

Signal Processing

Project 2

Submitted by: Miriam Chavay

Submission date: 15.6.2026

Part 1

1. At this stage we built the time axis of the EEG signal. The total number of samples was calculated according to the recording duration and the sampling rate of the system: $N = T \times f_s = 40 \times 1024 = 40,960$. That is, 40,960 samples were obtained for a recording 40 seconds long and at a sampling rate of 1024 Hz. Using `np.linspace`, a time vector was created with constant sampling intervals:

$$\Delta t = \frac{1}{f_s} = \frac{1}{1024} \approx 0.976 \text{ ms}$$

In addition, `endpoint=False` was set in order to match the number of time points exactly to the number of samples in the data files and to preserve the accuracy of the analysis.

Code display:

```
fs = 1024
duration = 40
N = duration * fs

time_vector = np.linspace(0, duration, N, endpoint=False)
print(f"Length of time vector: {len(time_vector)}")
```

Length of time vector: 40960

2. For an initial comparison between the EEG signals in the eyes-open (EO) and eyes-closed (EC) states, a 2-second time window was chosen, between the 10th and the 12th second of the recording. This window makes it possible to display the structure of the signal and its oscillations more clearly, without the clutter resulting from a large number of samples.

In the code we separated the EEG signals of the two states for the three participants. We then defined the start and end indices of the time window using the sampling rate ($f_s = 1024$ Hz), according to the relation between time and sample number. For each participant, the signal segment corresponding to the chosen time window was extracted, and the signals were displayed in separate graphs, with the EO and EC signals shown together for a direct comparison of the electrical activity over time.

Code display:

```
ec_504 = data_504[:, 0]
eo_504 = data_504[:, 1]

ec_519 = data_519[:, 0]
eo_519 = data_519[:, 1]

ec_524 = data_524[:, 0]
eo_524 = data_524[:, 1]

start_sec = 10
end_sec = 12

start_idx = int(start_sec * fs)
end_idx = int(end_sec * fs)

fig, axes = plt.subplots(3, 1, figsize=(14, 12), sharex=True)

participants = ['504', '519', '524']
eo_signals = [eo_504, eo_519, eo_524]
ec_signals = [ec_504, ec_519, ec_524]
colors = {'EO': '#1f77b4', 'EC': '#d62728'}

for i, ax in enumerate(axes):
    eo_segment = eo_signals[i][start_idx:end_idx]
    ec_segment = ec_signals[i][start_idx:end_idx]
    time_segment = time_vector[start_idx:end_idx]

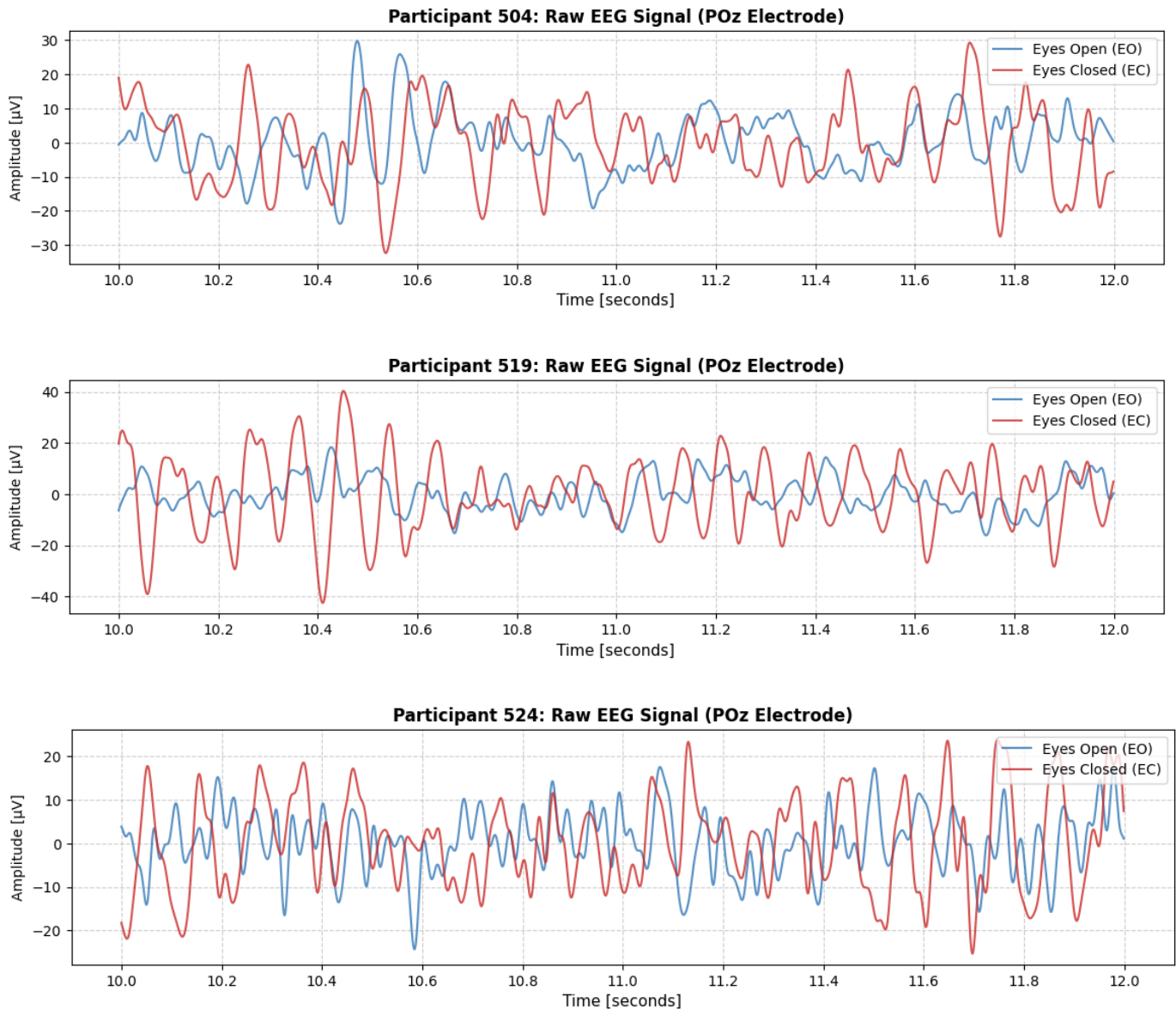
    ax.plot(time_segment, eo_segment, label='Eyes Open (EO)', color=colors['EO'], alpha=0.8, linewidth=1.5)
    ax.plot(time_segment, ec_segment, label='Eyes Closed (EC)', color=colors['EC'], alpha=0.8, linewidth=1.5)

    ax.set_title(f'Participant {participants[i]}: Raw EEG Signal (P0z Electrode)', fontsize=12, fontweight='bold')
    ax.set_ylabel('Amplitude [µV]', fontsize=10)
    ax.legend(loc='upper right')
    ax.grid(True, linestyle='--', alpha=0.6)
    ax.set_xlabel('Time [seconds]', fontsize=11)
    ax.set_xticks(np.arange(10, 12.1, 0.2))
    ax.tick_params(axis='x', labelbottom=True)
    plt.subplots_adjust(hspace=0.5)

plt.suptitle(f'Raw EEG Comparison between EO and EC ({start_sec}-{end_sec} Seconds Window)', fontsize=14, fontweight='bold', y=0.95)
plt.show()
```

3. According to the results of the graphs, the signal in the EC state appears more periodic and regular, with oscillations of relatively high amplitude, whereas in the EO state the signal is less regular and has faster activity. In addition, in the EC state clear alpha oscillations can be observed, i.e., periodic oscillations at a frequency of about 8–12 Hz. According to the number of cycles seen over the time window, a rate of about 10 oscillations per second is obtained, which matches the rhythm typical of eye closure. The phenomenon was observed in all the participants, although with different intensities from one to another.

Raw EEG Comparison between EO and EC (10-12 Seconds Window)



Part 2

1. In this section, we applied a Band-Pass filter in the frequency range 2-4 HZ. First, a Butterworth filter of order 5 was built using the function `butter`, where the cutoff frequencies were normalized relative to the Nyquist frequency. The filter was then applied to the signal using the function `filtfilt`, which performs forward and backward filtering and thereby prevents phase distortions. This filtering makes it possible to preserve the frequency range relevant to EEG activity and to reduce noise and frequencies. Code display:

```
def butter_bandpass(lowcut, highcut, fs, order=5):
    nyquist = 0.5 * fs
    low = lowcut / nyquist
    high = highcut / nyquist
    b, a = butter(order, [low, high], btype='band')
    return b, a

def apply_bandpass_filter(signal, lowcut=2, highcut=40, fs=1024, order=5):
    b, a = butter_bandpass(lowcut, highcut, fs, order)
    return filtfilt(b, a, signal)
```

```
eo_504 = np.nan_to_num(eo_504)
ec_504 = np.nan_to_num(ec_504)

eo_519 = np.nan_to_num(eo_519)
ec_519 = np.nan_to_num(ec_519)

eo_524 = np.nan_to_num(eo_524)
ec_524 = np.nan_to_num(ec_524)

eo_504_filt = apply_bandpass_filter(eo_504, fs=fs)
ec_504_filt = apply_bandpass_filter(ec_504, fs=fs)

eo_519_filt = apply_bandpass_filter(eo_519, fs=fs)
ec_519_filt = apply_bandpass_filter(ec_519, fs=fs)

eo_524_filt = apply_bandpass_filter(eo_524, fs=fs)
ec_524_filt = apply_bandpass_filter(ec_524, fs=fs)
```

2. Before applying the filter, missing values were handled using the function `np.nan_to_num`, which replaces NaN values with numerical values. This step is required in order to prevent calculation errors during the filtering process. The Band-Pass filter in the range 2-4 HZ was then applied to each of the EO and EC signals of the three participants using the function `apply_bandpass_filter`. The result is a filtered version of each signal, containing mainly the frequency range relevant to the analysis of EEG activity.

3. In order to evaluate the effect of the filtering, a 2-second time window from the recording was chosen. For each participant, the EO and EC conditions were displayed separately, with the raw signal and the filtered signal plotted on the same graph. The raw signal was displayed in gray, and the filtered signal EO in blue and EC in red.

Comparing the raw signal to the filtered signal shows that the Band-Pass filter reduced noise and frequency components that are not relevant to the EEG analysis. After the filtering the signal looks smoother and cleaner, while the main oscillations of the brain activity were preserved. In addition, it can be seen that the peaks appear at the same times both before and after the filtering, which indicates that the filtering did not change the timing of the signal but only removed unwanted noise components.

Code and results display:

```
participants = ['504', '519', '524']
raw_eo = [eo_504, eo_519, eo_524]
raw_ec = [ec_504, ec_519, ec_524]
filt_eo = [eo_504_filt, eo_519_filt, eo_524_filt]
filt_ec = [ec_504_filt, ec_519_filt, ec_524_filt]

time_segment = time_vector[start_idx:end_idx]

for i in range(3):
    fig, axes = plt.subplots(2, 1, figsize=(14, 8), sharex=True)
    raw_eo_seg = raw_eo[i][start_idx:end_idx]
    filt_eo_seg = filt_eo[i][start_idx:end_idx]

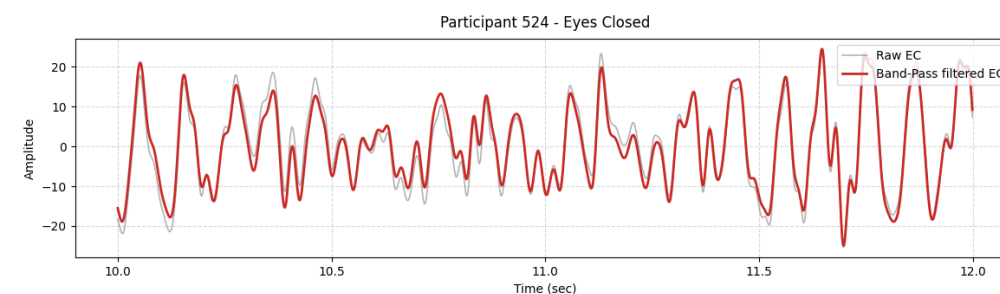
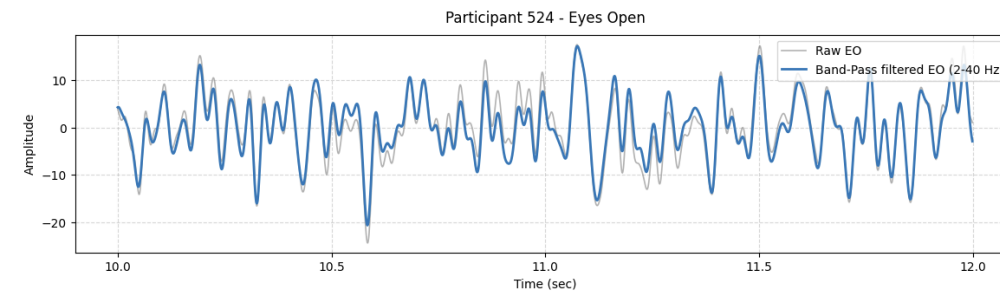
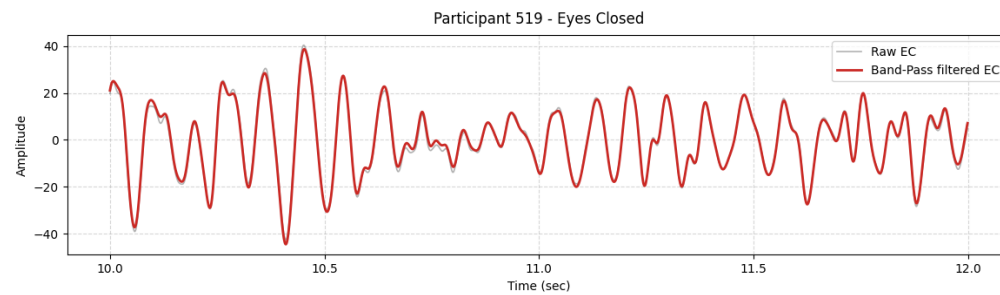
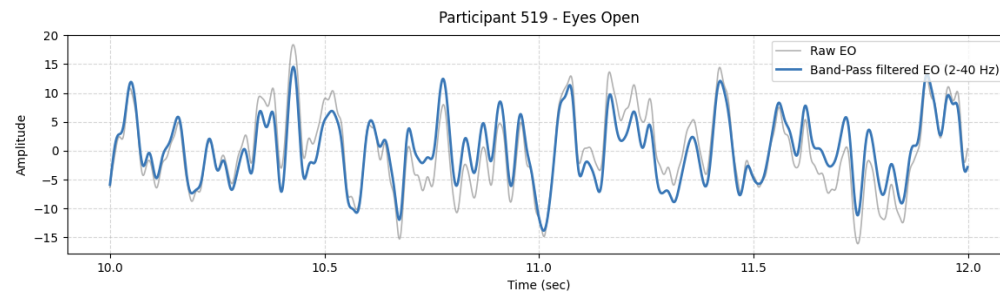
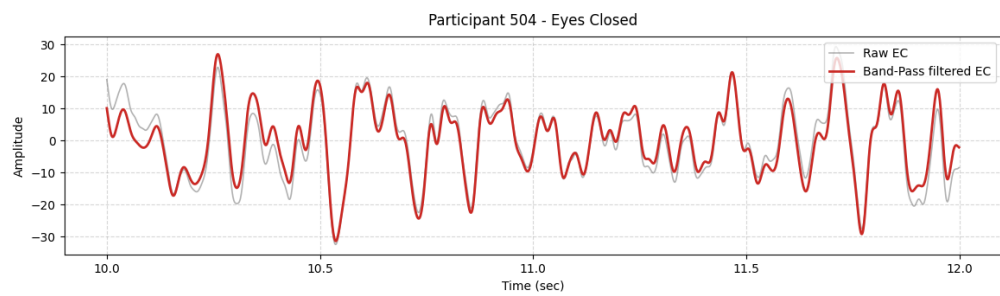
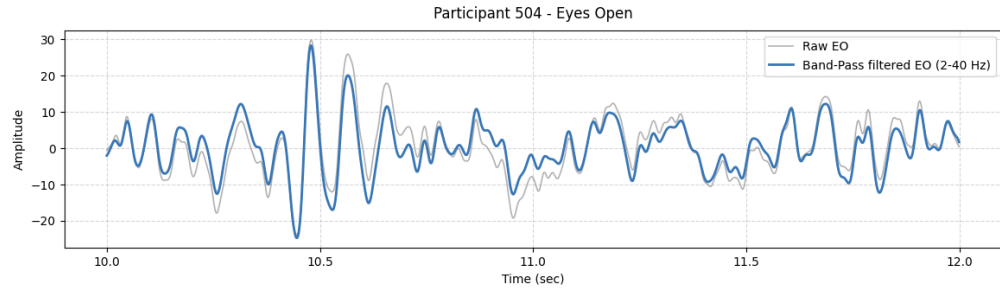
    raw_ec_seg = raw_ec[i][start_idx:end_idx]
    filt_ec_seg = filt_ec[i][start_idx:end_idx]
```

```

axes[0].plot(time_segment, raw_eo_seg, label='Raw EO', color='grey', alpha=0.6, linewidth=1.2)
axes[0].plot(time_segment, filt_eo_seg, label='Band-Pass filtered EO (2-40 Hz)', color='#1f77b4', linewidth=2)
axes[0].set_title(f'Participant {participants[i]} - Eyes Open')
axes[0].set_ylabel('Amplitude')
axes[0].legend(loc='upper right')
axes[0].grid(True, linestyle='--', alpha=0.5)

axes[1].plot(time_segment, raw_ec_seg, label='Raw EC', color='grey', alpha=0.6, linewidth=1.2)
axes[1].plot(time_segment, filt_ec_seg, label='Band-Pass filtered EC', color='#d62728', linewidth=2)
axes[1].set_title(f'Participant {participants[i]} - Eyes Closed')
axes[1].set_xlabel('Time (sec)')
axes[1].set_ylabel('Amplitude')
axes[1].legend(loc='upper right')
axes[1].grid(True, linestyle='--', alpha=0.5)

```



Part 3

1. For the spectral analysis of the EEG signals, the function `compute_fft_power_spectrum` was written, which receives a filtered signal and a sampling rate and calculates its power spectrum using FFT. First, the FFT of the signal was calculated using `np.fft.rfft`, which returns only the positive frequency components. Then the amplitude spectrum was calculated using the absolute value and normalization relative to the length of the signal, and the power spectrum was obtained by squaring the amplitude. In addition, the frequency vector corresponding to each spectral component was calculated. The analysis was performed for the filtered signals of both states for each of the three participants, and the results were displayed in graphs in the frequency range 0–50 Hz.

Code display:

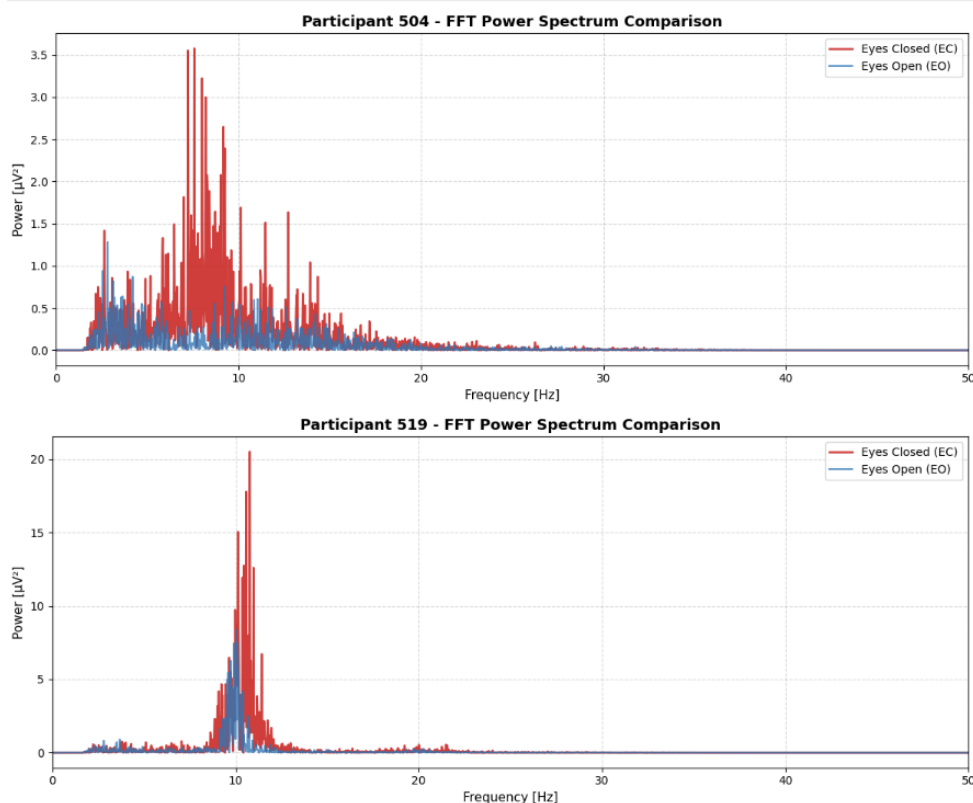
```
def compute_fft_power_spectrum(signal, fs):
    N = len(signal)
    fft_values = np.fft.rfft(signal)
    amplitude_spectrum = np.abs(fft_values) / (N / 2)
    amplitude_spectrum[0] = np.abs(fft_values[0]) / N
    power_spectrum = amplitude_spectrum ** 2
    frequencies = np.fft.rfftfreq(N, d=1/fs)
    return frequencies, power_spectrum

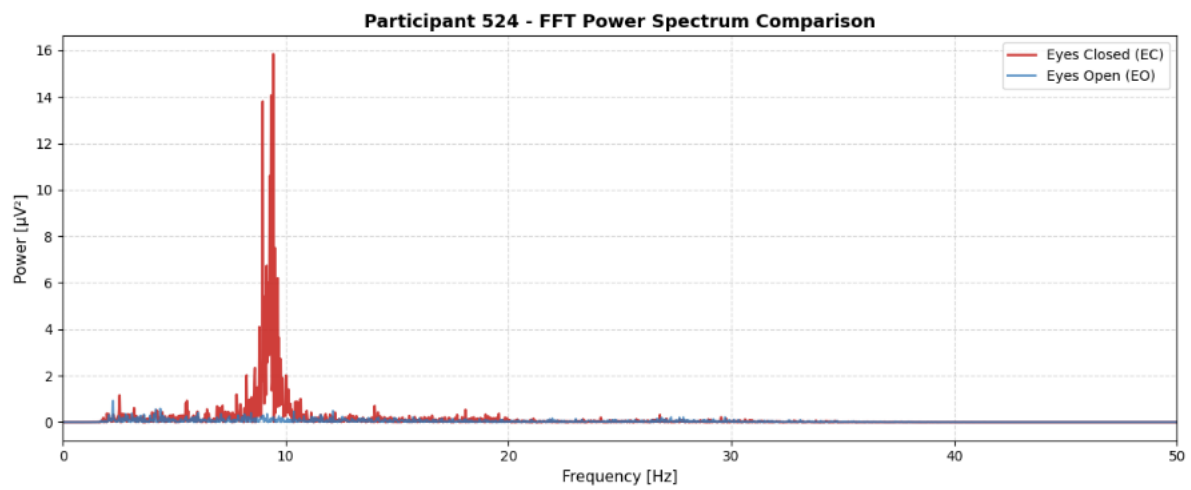
participants = ['504', '519', '524']
filtered_eo_signals = [eo_504_filt, eo_519_filt, eo_524_filt]
filtered_ec_signals = [ec_504_filt, ec_519_filt, ec_524_filt]

for i in range(len(participants)):

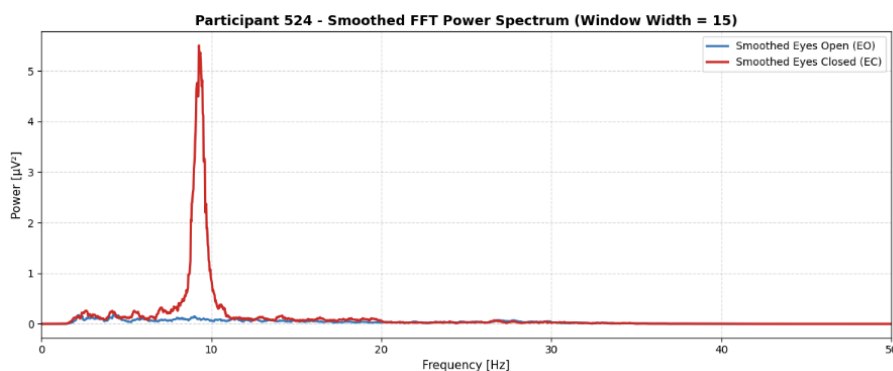
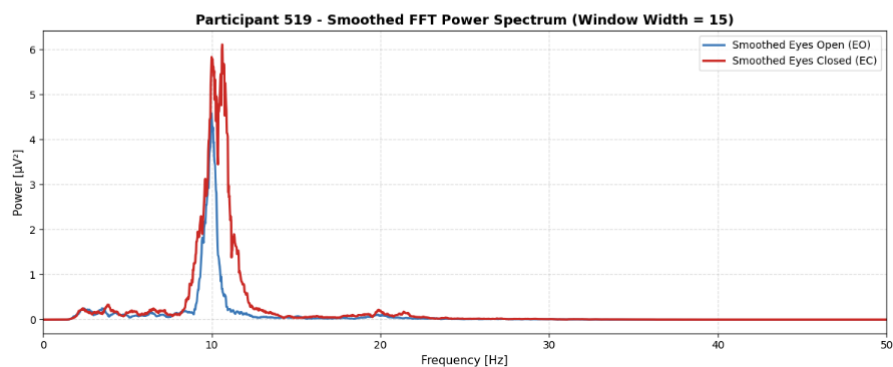
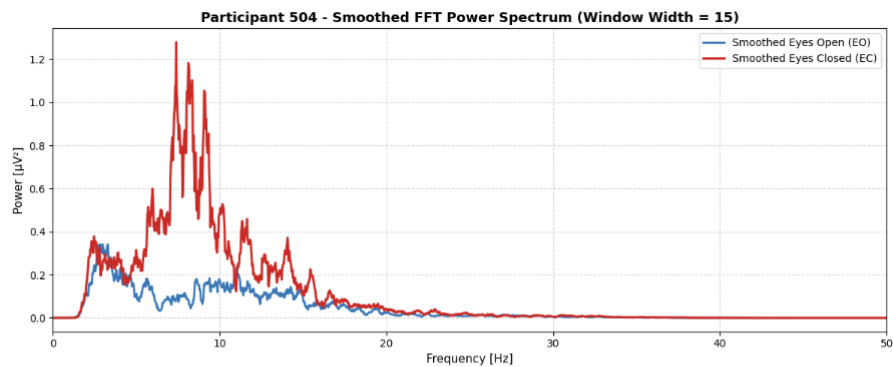
    freq_eo, power_eo = compute_fft_power_spectrum(filtered_eo_signals[i], fs)
    freq_ec, power_ec = compute_fft_power_spectrum(filtered_ec_signals[i], fs)
```

From the graphs obtained, the power distribution can be compared between the two states. In the eyes-closed (EC) state a more prominent peak is usually observed in the alpha frequency range (8–12 Hz), indicating strong periodic activity in this region. In contrast, in the eyes-open (EO) state the power of the activity in the alpha range is lower and the spectrum appears more spread out over various frequency ranges. This result is consistent with the facts regarding the appearance of the alpha rhythm during rest with eyes closed and its suppression when the eyes are opened. Graph display:





2. After calculating the power spectrum using FFT, smoothing was performed along the frequency axis in order to reduce random fluctuations and noise. For this purpose the Moving Average Smoothing method was used, implemented through convolution (`np.convolve`) between the power spectrum and a normalized uniform Rectangular Window, such that the sum of its coefficients equals 1. The smoothing parameter chosen was a window 15 frequency points wide (15 frequency bins). For a signal length of 40 seconds, the frequency resolution of the FFT is 0.025 Hz, and therefore the smoothing window covers a frequency width of 0.375 Hz. This width provides a balance between noise reduction and preservation of the main characteristics of the spectrum, and in particular the location and general shape of the alpha peak. A comparison between the raw FFT spectrum and the smoothed spectrum shows that the smoothing reduces sharp fluctuations and presents a continuous and cleaner curve, which enables clear identification of the dominant frequency ranges. In particular, the alpha peak in the EC condition stands out more clearly after smoothing, which improves the ability to perform accurate identification and analysis of the Individual Alpha Frequency (IAF). Code and results display:



```
def compute_smoothed_fft_spectrum(signal, fs, window_size=15):
    N = len(signal)
    fft_values = np.fft.rfft(signal)
    amplitude_spectrum = np.abs(fft_values) / (N / 2)
    amplitude_spectrum[0] = np.abs(fft_values[0]) / N
    power_spectrum = amplitude_spectrum ** 2
    kernel = np.ones(window_size) / window_size
    smoothed_power = np.convolve(power_spectrum, kernel, mode='same')
    frequencies = np.fft.rfftfreq(N, d=1/fs)
    return frequencies, smoothed_power

participants = ['504', '519', '524']
filtered_eo_signals = [eo_504_filt, eo_519_filt, eo_524_filt]
filtered_ec_signals = [ec_504_filt, ec_519_filt, ec_524_filt]
smoothing_window = 15

for i in range(len(participants)):
    freq_eo, power_eo_smooth = compute_smoothed_fft_spectrum(filtered_eo_signals[i], fs, window_size=smoothing_window)
    freq_ec, power_ec_smooth = compute_smoothed_fft_spectrum(filtered_ec_signals[i], fs, window_size=smoothing_window)
```

3. For the PSD evaluation, the Welch method was used, which improves the stability of the spectrum estimate by dividing the signal into segments, calculating a spectrum for each segment, and averaging the results. In this analysis a Hann window with a length of 2048 samples and an overlap of 1024 samples between consecutive segments was chosen. The Hann window was chosen because it reduces the Spectral Leakage phenomenon caused by the finite truncation of the signal, and thus allows a more accurate representation of the dominant frequency components. The window length chosen is equivalent to 2 seconds at a sampling rate of 1024 Hz, and provides a frequency resolution of:

$$\Delta f = \frac{f_s}{N} = \frac{1024}{2048} = 0.5 \text{ Hz}$$

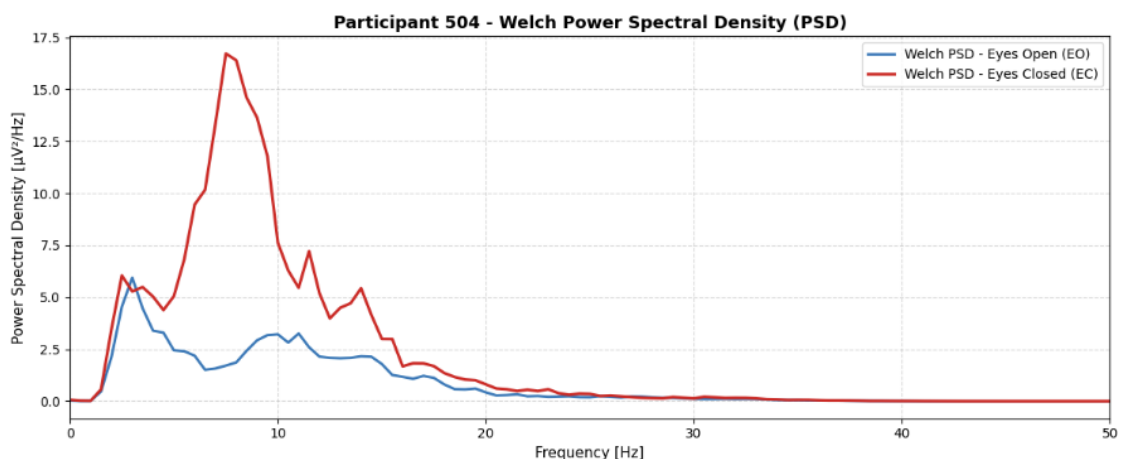
This resolution provides a good balance between sufficient frequency separation and a large enough number of segments for reliable averaging of the spectrum. In addition, a 50% overlap was chosen, since it increases the number of segments participating in the averaging and reduces the variance of the PSD estimate while preserving continuity of information between the segments. The combination of these parameters yields a smoother and more stable PSD estimate compared to a single FFT, and enables clear identification of the main spectral components, and in particular of the alpha peak in the EC condition. Code and results display:

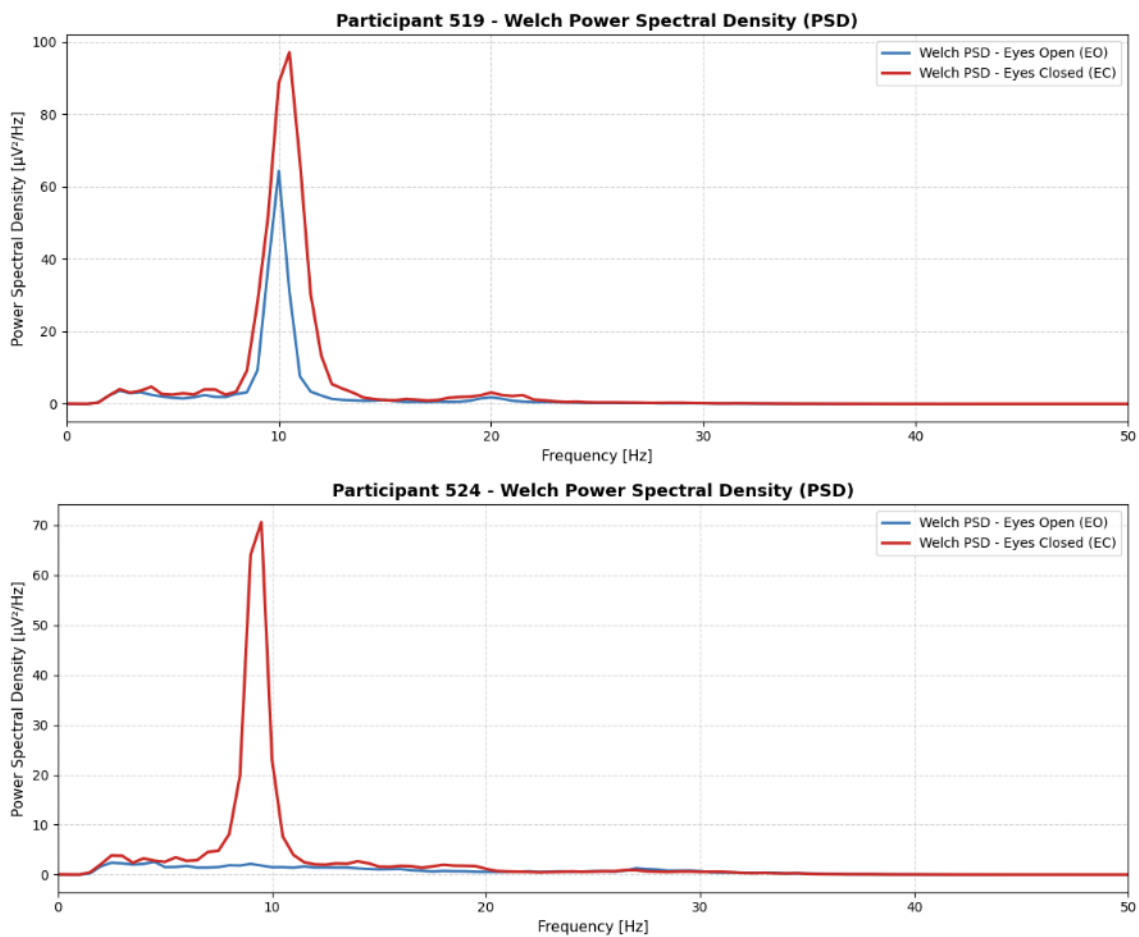
```
def compute_welch_psd(signal, fs, nperseg=2048, noverlap=1024, window='hann'):
    frequencies, psd = welch(signal, fs=fs, window=window,
                             nperseg=nperseg, noverlap=noverlap, scaling='density')
    return frequencies, psd

participants = ['504', '519', '524']
filtered_eo_signals = [eo_504_filt, eo_519_filt, eo_524_filt]
filtered_ec_signals = [ec_504_filt, ec_519_filt, ec_524_filt]

segment_length = 2048
overlap_points = 1024
window_type = 'hann'

for i in range(len(participants)):
    freq_eo, psd_eo = compute_welch_psd(filtered_eo_signals[i], fs,
                                         nperseg=segment_length, noverlap=overlap_points, window=window_type)
    freq_ec, psd_ec = compute_welch_psd(filtered_ec_signals[i], fs,
                                         nperseg=segment_length, noverlap=overlap_points, window=window_type)
```





4. To evaluate the IAF we implemented a function that locates the frequency with the maximum power in the alpha range, which was defined as 7.5–12.5 Hz. First, all the frequency points belonging to this range were selected using a logical mask, and then the function `np.argmax` was used to locate the frequency corresponding to the highest power. The function was applied to each of the three participants, in both the EO and EC conditions, and for all three methods: raw FFT, smoothed FFT and Welch PSD. Code and results display:

```
def estimate_iaf(frequencies, power_spectrum, f_min=7.5, f_max=12.5):
    alpha_mask = (frequencies >= f_min) & (frequencies <= f_max)
    alpha_freqs = frequencies[alpha_mask]
    alpha_powers = power_spectrum[alpha_mask]
    max_power_idx = np.argmax(alpha_powers)
    iaf = alpha_freqs[max_power_idx]
    return iaf

participants = ['504', '519', '524']
filtered_eo = [eo_504_filt, eo_519_filt, eo_524_filt]
filtered_ec = [ec_504_filt, ec_519_filt, ec_524_filt]

results_records = []

for i, sub_id in enumerate(participants):
    f_raw, p_raw_eo = compute_fft_power_spectrum(filtered_eo[i], fs)
    _, p_raw_ec = compute_fft_power_spectrum(filtered_ec[i], fs)

    iaf_raw_eo = estimate_iaf(f_raw, p_raw_eo)
    iaf_raw_ec = estimate_iaf(f_raw, p_raw_ec)

    f_smooth, p_smooth_eo = compute_smoothed_fft_spectrum(filtered_eo[i], fs, window_size=15)
    _, p_smooth_ec = compute_smoothed_fft_spectrum(filtered_ec[i], fs, window_size=15)

    iaf_smooth_eo = estimate_iaf(f_smooth, p_smooth_eo)
    iaf_smooth_ec = estimate_iaf(f_smooth, p_smooth_ec)

    f_welch, p_welch_eo = compute_welch_psd(filtered_eo[i], fs, nperseg=2048, noverlap=1024, window='hann')
    _, p_welch_ec = compute_welch_psd(filtered_ec[i], fs, nperseg=2048, noverlap=1024, window='hann')

    iaf_welch_eo = estimate_iaf(f_welch, p_welch_eo)
    iaf_welch_ec = estimate_iaf(f_welch, p_welch_ec)
```

	Participant	Condition	Raw FFT [Hz]	Smoothed FFT [Hz]	Welch PSD [Hz]
0	504	EO (Eyes Open)	9.224550	11.024462	11.0
1	504	EC (Eyes Closed)	7.574630	8.124603	7.5
2	519	EO (Eyes Open)	10.024511	9.999512	10.0
3	519	EC (Eyes Closed)	10.774474	10.624481	10.5
4	524	EO (Eyes Open)	12.124408	8.999561	9.0
5	524	EC (Eyes Closed)	9.449539	9.274547	9.5

The IAF results obtained were all found within the expected alpha range (about 7.5–12.5 Hz). In most cases similar values were obtained between the smoothed FFT method and the Welch method, which indicates higher stability and consistency compared to the raw FFT. It can be seen that for some of the participants there are certain differences between the EO and EC conditions, but these differences are not consistent across all the subjects. This finding is expected, since the IAF represents the dominant frequency of the alpha activity and not its power. In contrast, the raw FFT sometimes showed larger deviations, most likely due to its sensitivity to noise and local peaks. In general, the Welch method provided the most stable and reliable estimates of the individual alpha frequency (IAF), and was therefore chosen for use in the following stages of the analysis.

Results and Discussion:

To analyze the spectral activity of the EEG signals we calculated three estimates: a power spectrum using FFT, a smoothed FFT spectrum using a Moving Average 15 frequency points wide, and a PSD using the Welch method with a Hann window 2048 samples long and an overlap of 50%. For each method, the EO and EC spectra were displayed, and the IAF was identified by locating the frequency with the maximum power in the range 7.5-12.5 Hz.

Comparing the methods, the raw FFT spectrum was found to be more sensitive to noise and to display narrow, oscillatory peaks. In contrast, the smoothed FFT produces a more continuous and clearer curve, whereas the Welch method provides the smoothest and most stable spectrum thanks to dividing the signal into segments and averaging the results.

In all the participants, more prominent alpha activity was observed in the EC state compared to EO, as expected physiologically. In addition, differences were found between the participants in the location and power of the alpha peak, indicating variability between the subjects in brain activity. Comparing the IAF values between the methods showed that the smoothed FFT and the Welch method provide more stable estimates than the raw FFT, with the Welch method appearing to be the most reliable for identifying the alpha frequency.

In this section three graphs were displayed for each subject, comparing the three spectral estimation methods and marking the IAF values obtained in each of them. Code display:

```

colors = {'EO': '#1f77b4', 'EC': '#d62728'}
smoothing_window = 15
kernel = np.ones(smoothing_window) / smoothing_window

for i, sub_id in enumerate(participants):

    # FFT normalized
    f_raw, p_raw_eo = compute_fft_power_spectrum(filtered_eo[i], fs)
    _, p_raw_ec = compute_fft_power_spectrum(filtered_ec[i], fs)

    iaf_raw_eo = estimate_iaf(f_raw, p_raw_eo)
    iaf_raw_ec = estimate_iaf(f_raw, p_raw_ec)

    # FFT smoothed
    p_smooth_eo = np.convolve(p_raw_eo, kernel, mode='same')
    p_smooth_ec = np.convolve(p_raw_ec, kernel, mode='same')

    iaf_smooth_eo = estimate_iaf(f_raw, p_smooth_eo)
    iaf_smooth_ec = estimate_iaf(f_raw, p_smooth_ec)

    # Welch PSD
    f_welch, p_welch_eo = compute_welch_psd(filtered_eo[i], fs, nperseg=2048, noverlap=1024, window='hann')
    _, p_welch_ec = compute_welch_psd(filtered_ec[i], fs, nperseg=2048, noverlap=1024, window='hann')

    iaf_welch_eo = estimate_iaf(f_welch, p_welch_eo)
    iaf_welch_ec = estimate_iaf(f_welch, p_welch_ec)

fig, axes = plt.subplots(3, 1, figsize=(12, 13), sharex=True)

```

```

# Raw FFT Power Spectrum
axes[0].plot(f_raw, p_raw_eo, label=f'EO (IAF: {iaf_raw_eo:.2f} Hz)', color=colors['EO'], alpha=0.7, linewidth=1.2)
axes[0].plot(f_raw, p_raw_ec, label=f'EC (IAF: {iaf_raw_ec:.2f} Hz)', color=colors['EC'], alpha=0.8, linewidth=1.2)
axes[0].axvline(iaf_raw_eo, color=colors['EO'], linestyle='--', alpha=0.6)
axes[0].axvline(iaf_raw_ec, color=colors['EC'], linestyle='--', alpha=0.6)
axes[0].set_title('FFT Power Spectrum (Raw)', fontsize=11, fontweight='bold')
axes[0].set_ylabel('Power [ $\mu V^2$ ]', fontweight='bold')
axes[0].legend(loc='upper right')
axes[0].grid(True, linestyle='--', alpha=0.5)

# Smoothed FFT Spectrum
axes[1].plot(f_raw, p_smooth_eo, label=f'EO (IAF: {iaf_smooth_eo:.2f} Hz)', color=colors['EO'], linewidth=1.8)
axes[1].plot(f_raw, p_smooth_ec, label=f'EC (IAF: {iaf_smooth_ec:.2f} Hz)', color=colors['EC'], linewidth=2.0)
axes[1].axvline(iaf_smooth_eo, color=colors['EO'], linestyle='--', alpha=0.6)
axes[1].axvline(iaf_smooth_ec, color=colors['EC'], linestyle='--', alpha=0.6)
axes[1].set_title(f'Smoothed FFT Spectrum (Window Size = {smoothing_window})', fontsize=11, fontweight='bold')
axes[1].set_ylabel('Power [ $\mu V^2$ ]', fontweight='bold')
axes[1].legend(loc='upper right')
axes[1].grid(True, linestyle='--', alpha=0.5)

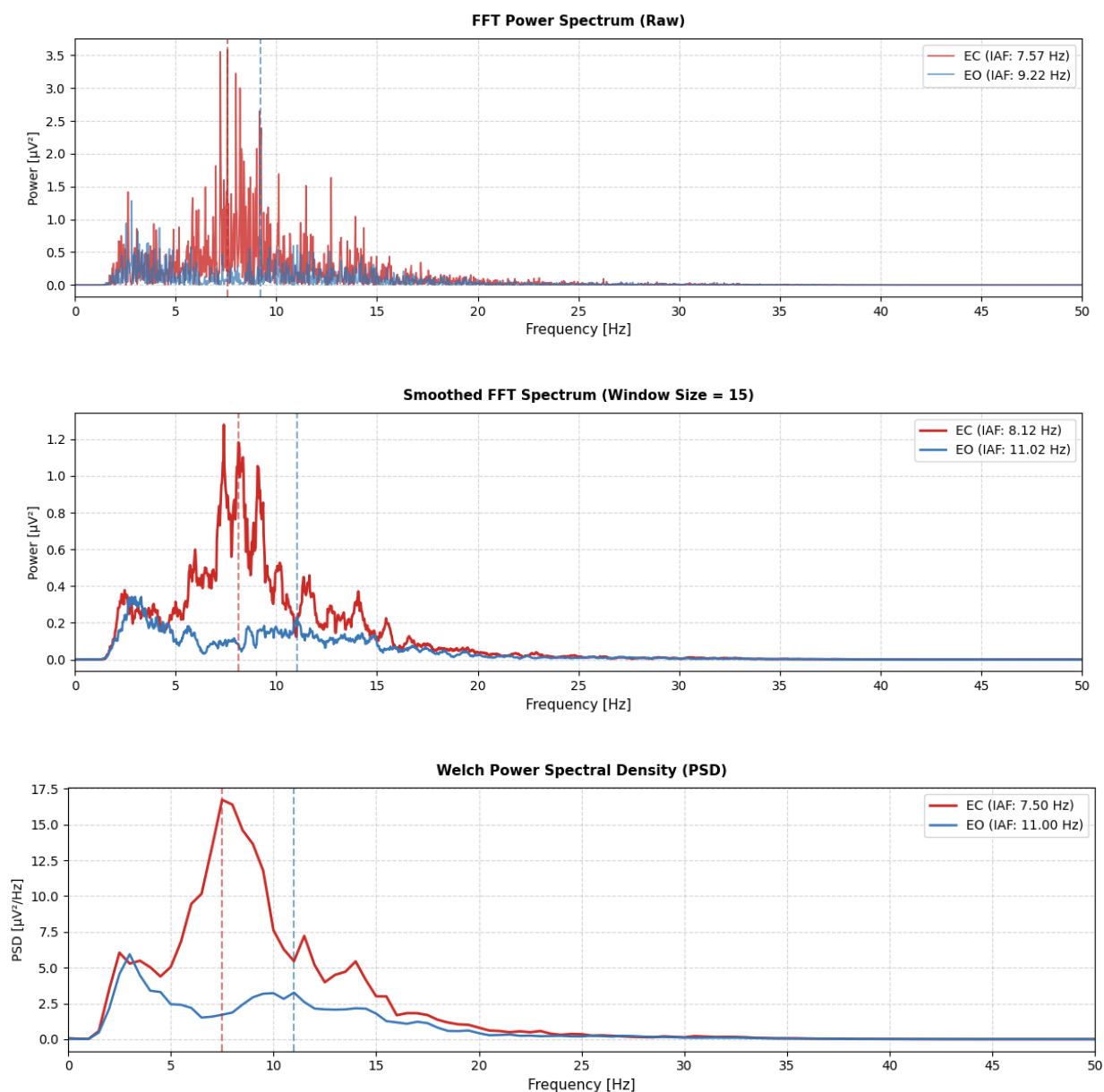
# Welch PSD
axes[2].plot(f_welch, p_welch_eo, label=f'EO (IAF: {iaf_welch_eo:.2f} Hz)', color=colors['EO'], linewidth=1.8)
axes[2].plot(f_welch, p_welch_ec, label=f'EC (IAF: {iaf_welch_ec:.2f} Hz)', color=colors['EC'], linewidth=2.0)
axes[2].axvline(iaf_welch_eo, color=colors['EO'], linestyle='--', alpha=0.6)
axes[2].axvline(iaf_welch_ec, color=colors['EC'], linestyle='--', alpha=0.6)
axes[2].set_title('Welch Power Spectral Density (PSD)', fontsize=11, fontweight='bold')
axes[2].set_xlabel('Frequency [Hz]', fontsize=11)
axes[2].set_ylabel('PSD [ $\mu V^2/Hz$ ]', fontweight='bold')
axes[2].legend(loc='upper right')
axes[2].grid(True, linestyle='--', alpha=0.5)

for ax in axes:
    ax.set_xlim(0, 50)

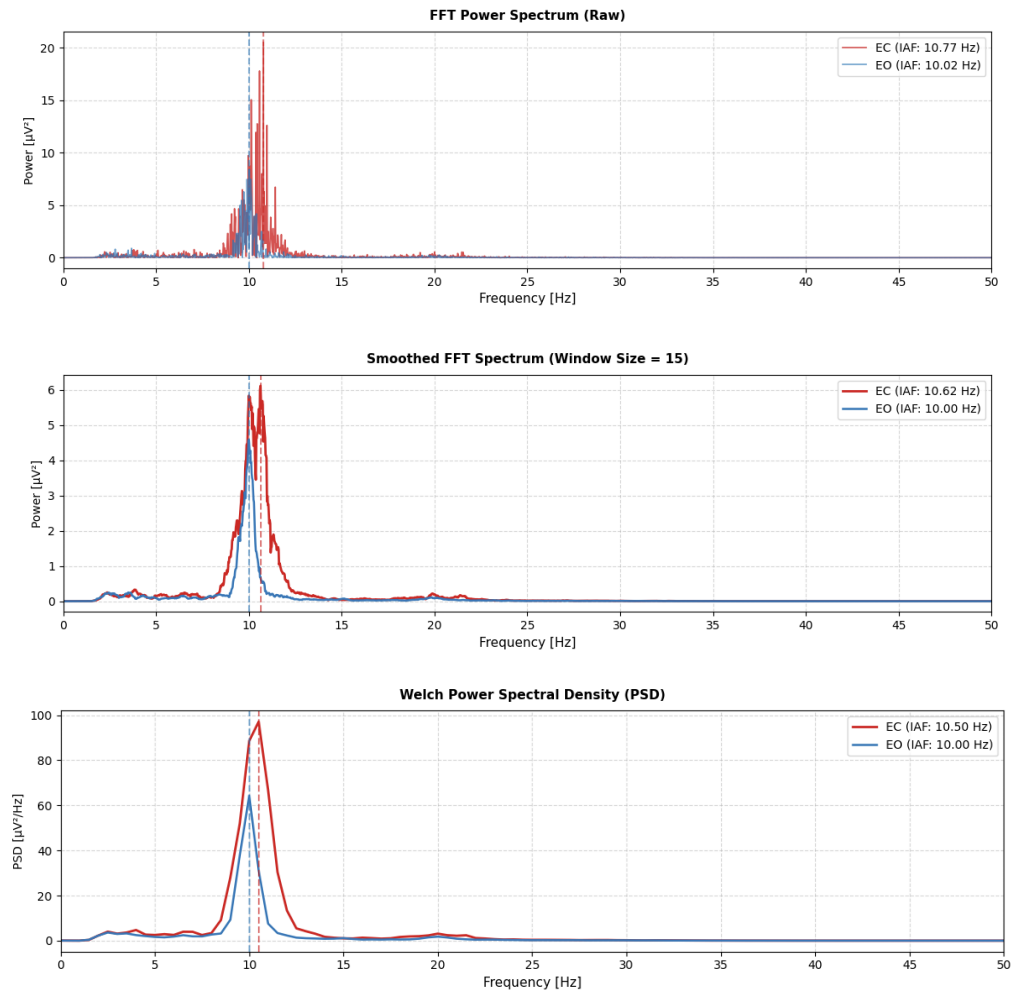
```

Graph display:

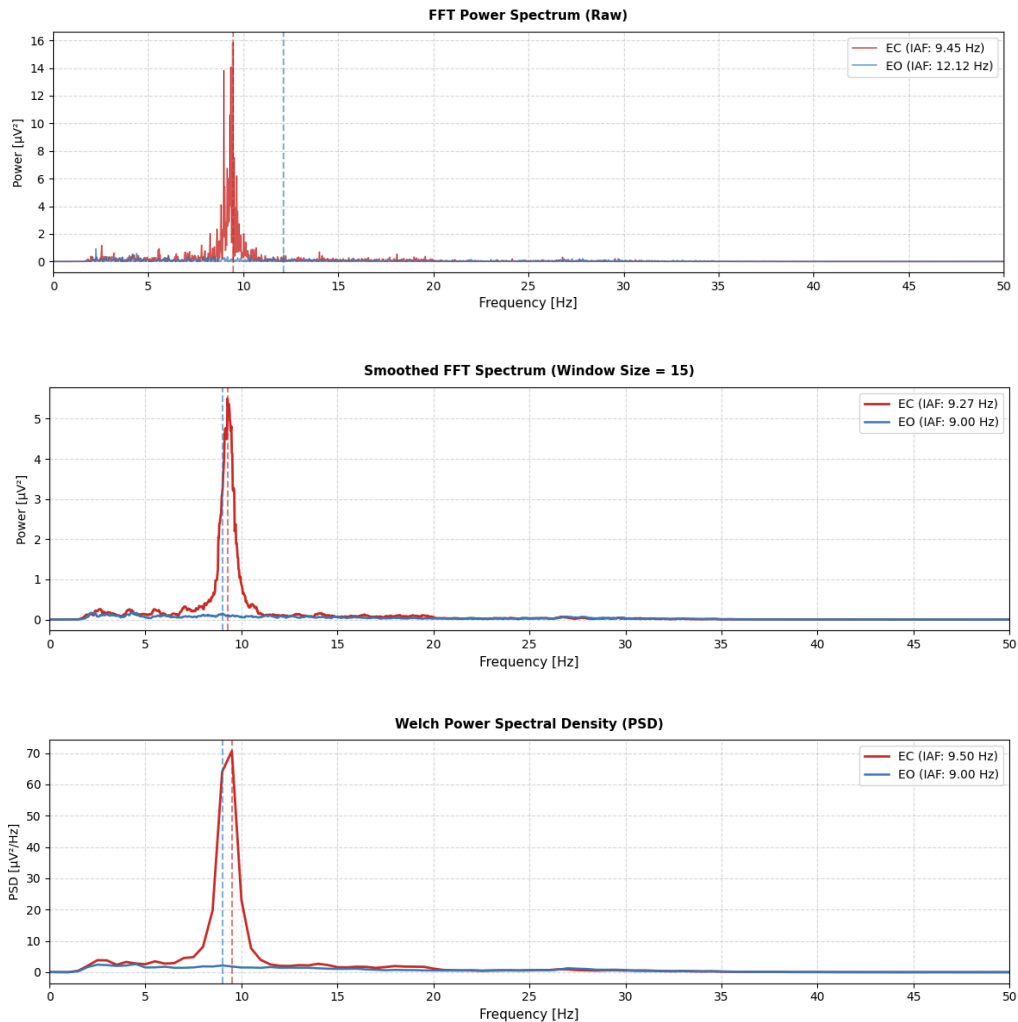
Participant 504 - Comprehensive Spectral Estimation Comparison



Participant 519 - Comprehensive Spectral Estimation Comparison



Participant 524 - Comprehensive Spectral Estimation Comparison



Part 4

1. The spectrogram was computed using the function `scipy.signal.spectrogram` for each of the three participants and separately for the Eyes Open (EO) and Eyes Closed (EC) conditions. For visual display, the PSD values were converted to dB units using: $\text{PSD}_{\text{dB}} = 10 \log_{10}(\text{PSD})$. The spectrograms were then displayed using `pcolormesh`, where the X axis represents time, the Y axis represents frequency, and the color represents the signal power. Code display:

```
window_type = 'hann'

for i, sub_id in enumerate(participants):
    fig, axes = plt.subplots(2, 1, figsize=(13, 10), sharex=True)

    f_eo, t_eo, Sxx_eo = spectrogram(filtered_eo_signals[i], fs=fs, window=window_type, nperseg=segment_length, noverlap=fs, scaling='density')

    Sxx_eo_db = 10 * np.log10(Sxx_eo + 1e-10)

    im0 = axes[0].pcolormesh(t_eo, f_eo, Sxx_eo_db, cmap='jet', shading='gouraud', vmin=-30, vmax=10)
    axes[0].set_title(f'Participant {sub_id} - Eyes Open (EO) Spectrogram', fontsize=12, fontweight='bold')
    axes[0].set_ylabel('Frequency [Hz]', fontsize=11)
    axes[0].set_ylim(0, 40)
    fig.colorbar(im0, ax=axes[0], label='Power Spectral Density [dB/Hz]')

    f_ec, t_ec, Sxx_ec = spectrogram(filtered_ec_signals[i], fs=fs, window=window_type, nperseg=segment_length, noverlap=fs, scaling='density')

    Sxx_ec_db = 10 * np.log10(Sxx_ec + 1e-10)

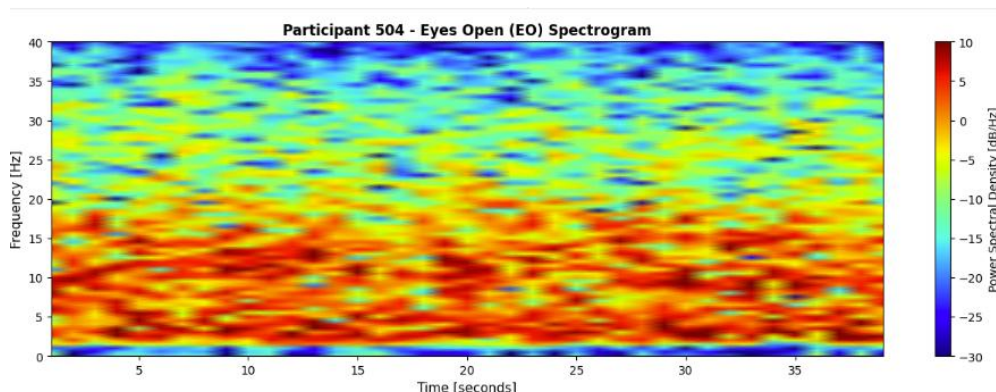
    im1 = axes[1].pcolormesh(t_ec, f_ec, Sxx_ec_db, cmap='jet', shading='gouraud', vmin=-30, vmax=10)
    axes[1].set_title(f'Participant {sub_id} - Eyes Closed (EC) Spectrogram', fontsize=12, fontweight='bold')
    axes[1].set_xlabel('Time [seconds]', fontsize=11)
    axes[1].set_ylabel('Frequency [Hz]', fontsize=11)
    axes[1].set_ylim(0, 40)
    fig.colorbar(im1, ax=axes[1], label='Power Spectral Density [dB/Hz]')
```

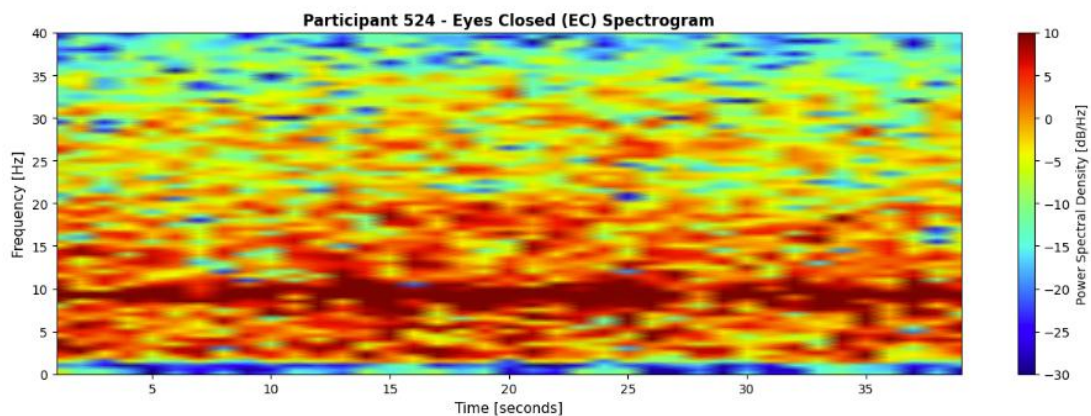
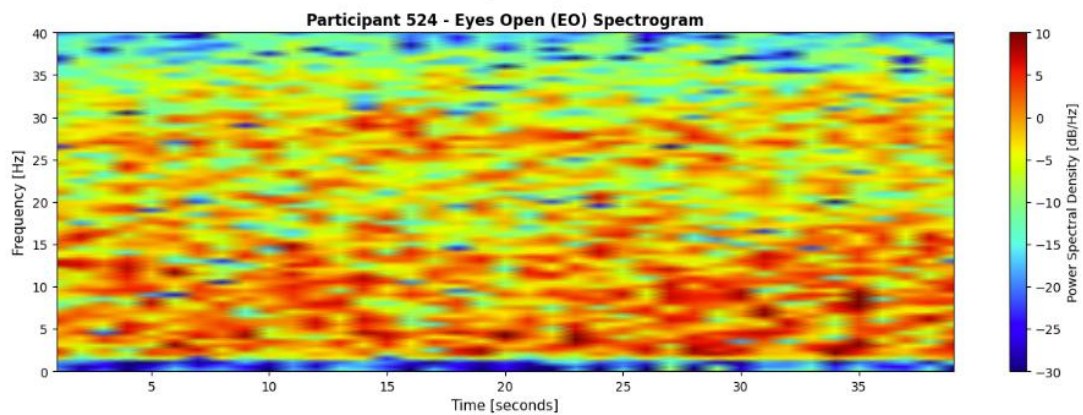
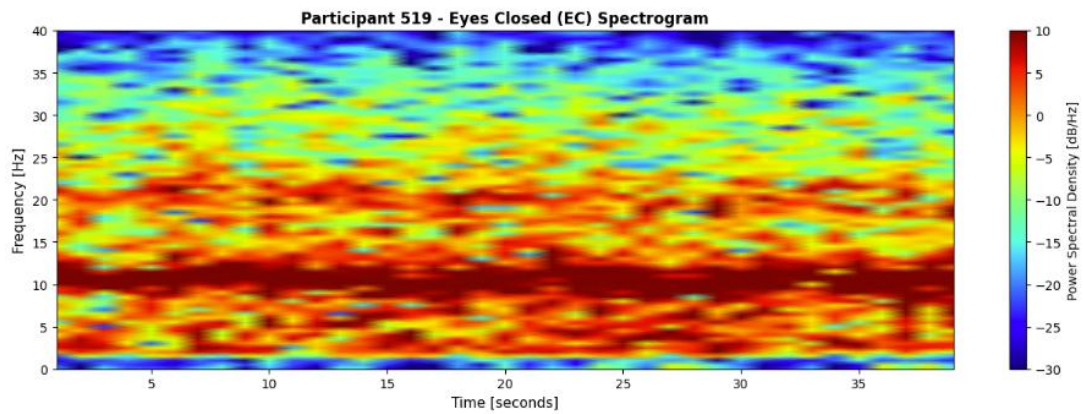
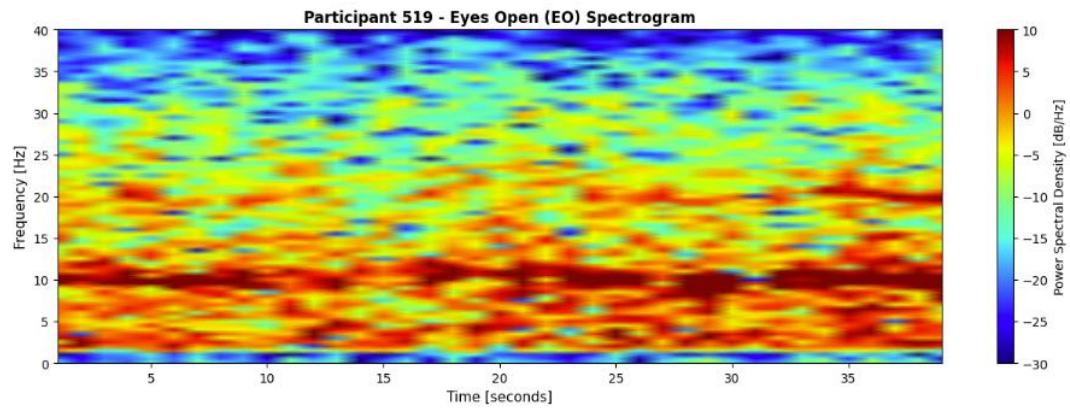
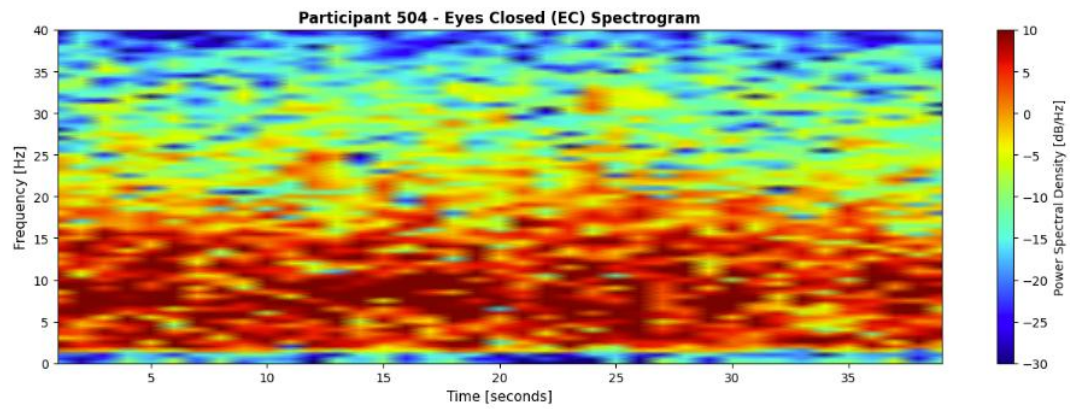
2. A spectrogram is a time–frequency representation of the signal. It is based on the short-time Fourier transform (STFT), in which the signal is divided into short time windows and an FFT is performed separately on each window. The result is a map showing how the spectral content of the signal changes over time. In the resulting graph the X axis represents time, the Y axis represents frequency, and the color represents the power of the signal (PSD). A regular power spectrum (FFT Power Spectrum) provides a global description of the frequencies present in the signal over the entire recording time, but does not provide information about when they appear. In contrast, a spectrogram also preserves temporal information and therefore makes it possible to identify changes in brain activity over time. For EEG signals, which are dynamic signals, a spectrogram provides a richer picture than the power spectrum alone.
3. In computing the spectrogram the following parameters were chosen: a Hann window with a window length of $nperseg = 2048$ and an overlap of $noverlap = 1024$, i.e., 50%. A window length of 2048 samples, at a sampling rate of 1024 Hz, represents a time window of 2 seconds and provides a frequency resolution of:

$$\Delta f = \frac{f_s}{N} = \frac{1024}{2048} = 0.5 \text{ Hz}$$

This resolution is sufficient for accurate identification of alpha activity in the range 8–12 Hz. The Hann window was chosen in order to reduce spectral leakage, while a 50% overlap enables a smooth transition between windows and reduces loss of information at the window edges.

4. For each participant a graph was created containing the two spectrograms, EO and EC. Displaying the two states in the same figure enables a direct comparison of brain activity between the conditions. Graph display:





5. The spectrograms show a consistent pattern in all the participants. In the EC (eyes closed) condition a clear and continuous energy band (dark red) is observed in the alpha range of about 8–12 Hz, indicating stable alpha activity over time. This activity appears as a continuous band of high energy throughout most of the recording.

In contrast, in the EO (eyes open) condition the power of the alpha activity is significantly lower, and the band in the alpha range is weaker or almost disappears. This finding is consistent with the Alpha Suppression phenomenon, in which opening the eyes causes a decrease in the synchronization of alpha waves.

In addition, differences were observed between the participants in the strength of the alpha band and in its exact location within the alpha range. These differences reflect variability in brain activity and reinforce the importance of using the individual alpha frequency (IAF) for each subject.

Part 5

1. To reconstruct the alpha activity, the IAF values obtained from the Welch PSD method were chosen. This choice is based on the findings of Part 3, where it was found that the Welch method provided the most stable and reliable estimate. The method reduces the spectral variance by dividing the signal into segments and averaging the results, and therefore allows a more accurate identification of the alpha peak compared to the raw FFT or the smoothed FFT.
- 2.–4. For each participant an individual alpha band was defined in the range: $IAF \pm 1 \text{ Hz}$. After performing an FFT on the signal, only the frequency components located in this range were kept and all other frequency components were set to zero. Finally, an inverse Fourier transform (IFFT) was performed to obtain the reconstructed time signal, which represents the alpha activity only. Code display:

```
def reconstruct_alpha_signal(signal, fs, iaf_estimate):
    N = len(signal)
    fft_values = np.fft.rfft(signal)
    frequencies = np.fft.rfftfreq(N, d=1/fs)
    low_bound = iaf_estimate - 1.0
    high_bound = iaf_estimate + 1.0
    alpha_mask = (frequencies >= low_bound) & (frequencies <= high_bound)
    filtered_fft_values = fft_values.copy()
    filtered_fft_values[~alpha_mask] = 0.0
    reconstructed_signal = np.fft.irfft(filtered_fft_values, n=N)
    return reconstructed_signal

iaf_values = {'504': 10.0, '519': 9.25, '524': 10.0}

start_sec, end_sec = 10, 12
start_idx = int(start_sec * fs)
end_idx = int(end_sec * fs)
time_segment = time_vector[start_idx:end_idx]

participants = ['504', '519', '524']
filtered_eo_signals = [eo_504_filt, eo_519_filt, eo_524_filt]
filtered_ec_signals = [ec_504_filt, ec_519_filt, ec_524_filt]

for i, sub_id in enumerate(participants):
    current_iaf = iaf_values[sub_id]

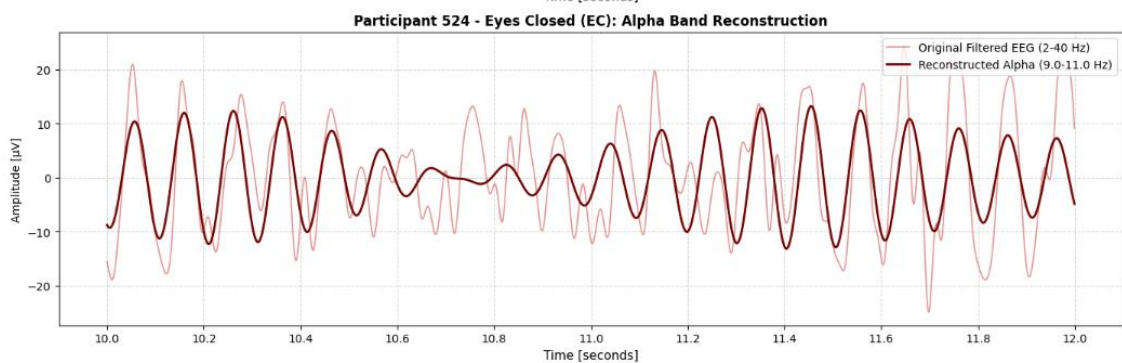
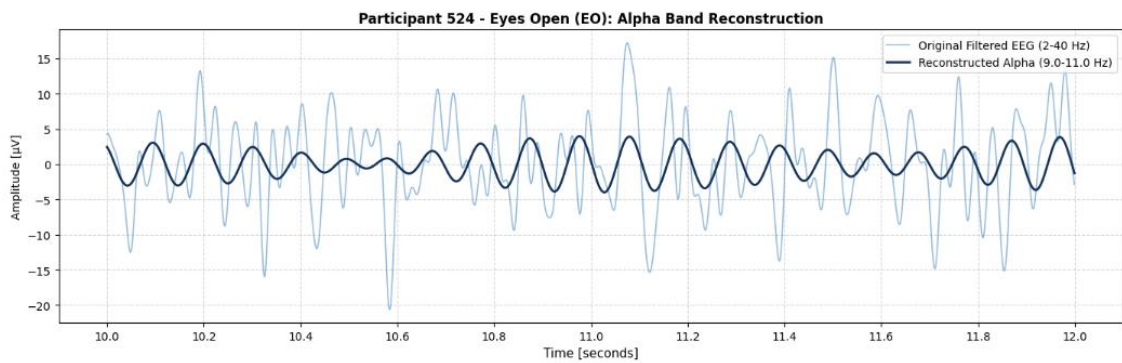
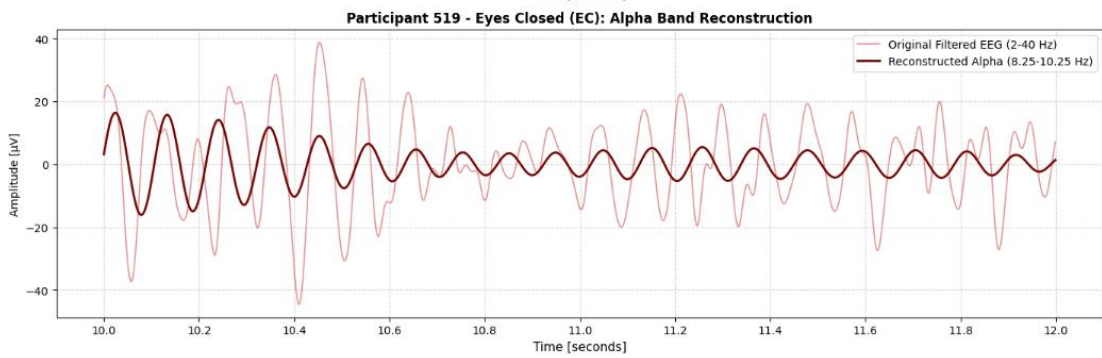
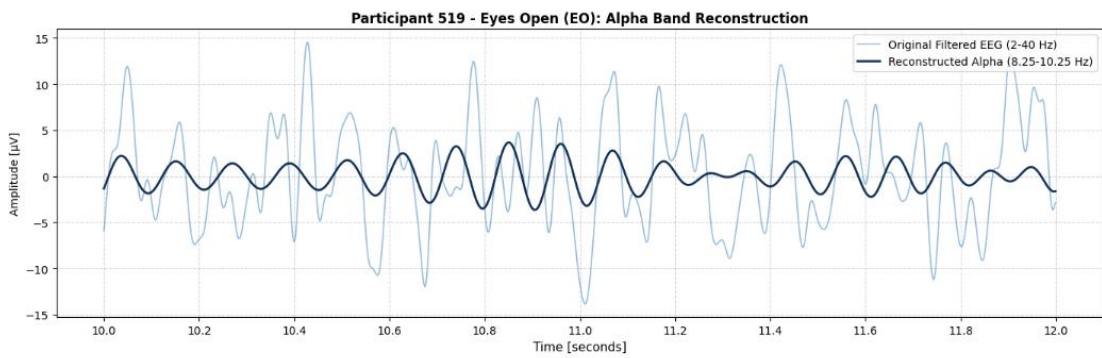
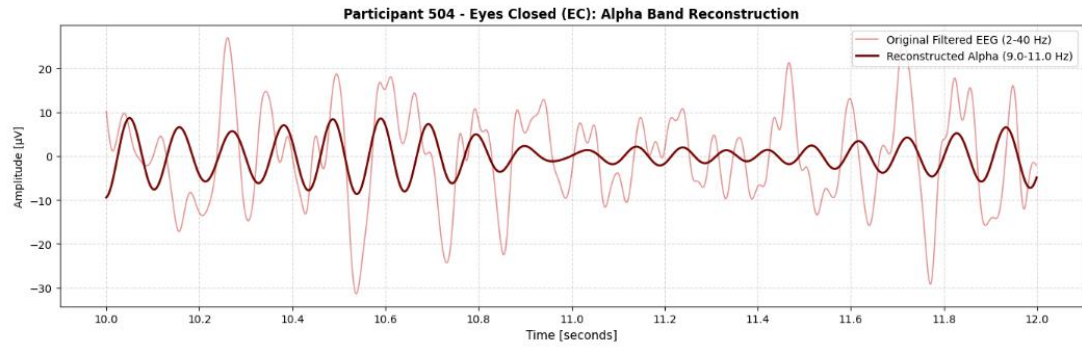
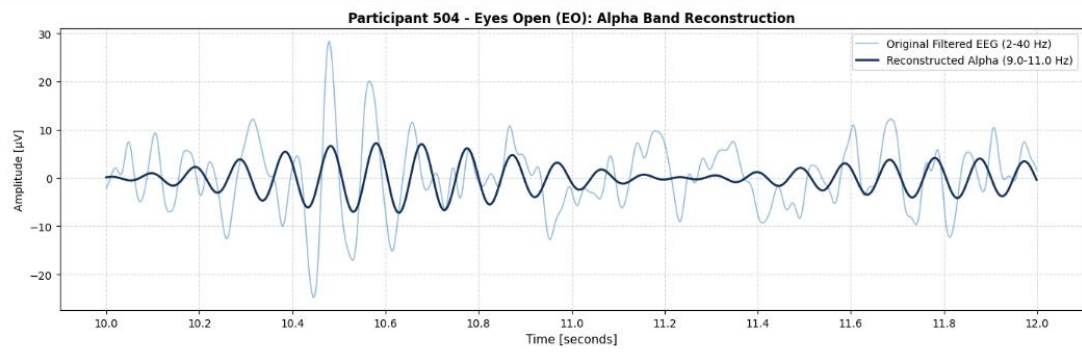
    alpha_eo_reconstructed = reconstruct_alpha_signal(filtered_eo_signals[i], fs, current_iaf)
    alpha_ec_reconstructed = reconstruct_alpha_signal(filtered_ec_signals[i], fs, current_iaf)

    fig, axes = plt.subplots(2, 1, figsize=(14, 9), sharex=True)

    axes[0].plot(time_segment, filtered_eo_signals[i][start_idx:end_idx],
                 label='Original Filtered EEG (2-40 Hz)', color='#1f77b4', alpha=0.5, linewidth=1.2)
    axes[0].plot(time_segment, alpha_eo_reconstructed[start_idx:end_idx],
                 label=f'Reconstructed Alpha ({current_iaf-1}-{current_iaf+1} Hz)', color='#002d54', linewidth=2)
    axes[0].set_title(f'Participant {sub_id} - Eyes Open (EO): Alpha Band Reconstruction', fontsize=12, fontweight='bold')
    axes[0].set_ylabel('Amplitude [μV]', fontsize=10)
    axes[0].legend(loc='upper right')
    axes[0].grid(True, linestyle='--', alpha=0.5)

    axes[1].plot(time_segment, filtered_ec_signals[i][start_idx:end_idx],
                 label='Original Filtered EEG (2-40 Hz)', color='#d62728', alpha=0.5, linewidth=1.2)
    axes[1].plot(time_segment, alpha_ec_reconstructed[start_idx:end_idx],
                 label=f'Reconstructed Alpha ({current_iaf-1}-{current_iaf+1} Hz)', color='#7a0000', linewidth=2)
    axes[1].set_title(f'Participant {sub_id} - Eyes Closed (EC): Alpha Band Reconstruction', fontsize=12, fontweight='bold')
    axes[1].set_xlabel('Time [seconds]', fontsize=11)
    axes[1].set_ylabel('Amplitude [μV]', fontsize=10)
    axes[1].legend(loc='upper right')
    axes[1].grid(True, linestyle='--', alpha=0.5)
```

5. For each participant two graphs were displayed, one for EO and the other for EC. In each graph the original filtered signal (2–40 Hz) and the signal reconstructed from the alpha band alone were compared, in a 2-second time window for a clear illustration of the oscillations of the signal. Graph display:



6. The original signal contains a variety of frequency ranges and noise components, while the reconstructed signal includes only the information found in the individual alpha band. The reconstruction process preserves the central alpha activity of the subject and removes other frequency components, such as activity in the theta and beta ranges, as well as some of the background noise. As a result, a simpler and clearer signal is obtained, which mainly represents the dominant alpha oscillation.

In addition, a clear difference was observed between the EO and EC conditions. In the eyes-closed (EC) state the reconstructed signal usually shows a higher amplitude, which indicates a strong presence of alpha activity. In contrast, in the eyes-open (EO) state the amplitude of the reconstructed signal is lower, in line with the Alpha Suppression phenomenon. This finding is consistent with the results obtained in the spectral analysis and in the spectrograms, and emphasizes the effect of the eye state on the strength of alpha activity in the brain.